Day 2 — What Data Are, and How to Write About Them

ECON 300 · Applied Statistics for Economists

Ari Kamau

2026-09-15

MASTER DECK. Every "You Try" slide is followed immediately by its solution slide. Present straight through; advance one slide to reveal the answer.

Press S for speaker notes, B for the chalkboard, ESC for the overview.

TIMING — this is the fullest day of the term. Protect the code hour. 0-10 News article 10-40 Cohen Ch. 2: what data are, measurement, sample vs. population 40-55 Reproducibility, .qmd anatomy, chunk options, math 55-60 You Try: first document 60-75 tidyverse setup, glimpse, the pipe 75-105 select / filter / mutate / summarise, with the four You Try blocks 105-115 ggplot2 115-120 Census pull; assign mini-exercise as homework if short

IF YOU FALL BEHIND: cut the geom_bar vs geom_col slides and the Census You Try, and assign both as reading. Do NOT cut the filter() block - the == mistake needs to happen in front of you.

This morning

Hour 1 · Theory

- What *are* data?
- Levels of measurement
- Accuracy, precision, resolution, range
- Sample vs. population
- Why reproducibility is a statistical issue
- Anatomy of a `.qmd` file

Hour 2 · Code

- Chunks, chunk options, math mode
- The pipe
- `select()` · `filter()` · `mutate()`
- `group_by()` + `summarise()`
- `ggplot2`
- A first real Census pull

Reading: Cohen Ch. 2. **Due Monday 9/21:** Problem Set 1.

Before we start — news

Today's article

Ten minutes. Three questions, and these are the same three questions every Tuesday for the rest of the term:

1. **What is the empirical claim?** State it as a sentence that could be false.
2. **What data would settle it?** Be specific — unit of observation, time period, what variable.
3. **What method, and what would break it?** Every method has a failure mode. Name it.

. . .

Keep a running list. One of these becomes your final project.

Hour 1 — What are data?

A deceptively hard question

Data are recorded observations. Every word there hides a decision.

. . .

- **Recorded** — by whom, with what instrument, under what incentive?
- **Observations** — of what unit? A person, a household, a quarter, a transaction?
- Something was measured, and everything *not* measured is now invisible to you forever.

. . .

> The single most common empirical mistake is treating the data as the world, rather than as one particular recording of it.

Where do data come from?

Source	Designed for	Danger
Survey	Answering your question	Non-response, self-report bias
Administrative	Running a program	Eligibility rules shape who appears
Transactional	Operating a business	Only records completed actions
Scraped	Nothing	Selection you cannot characterize
Simulated	Whatever you want	It is not the world

. . .

Most economic data are **administrative** — collected to run something, not to answer your question. HMDA exists because a law requires reporting, not because someone wanted a clean research dataset.

Levels of measurement

The type of a variable determines what arithmetic is *meaningful*, not just what R will let you do.

Level	Example	Meaningful	Not meaningful
Nominal	Census region	Counting, mode	Ordering, mean
Ordinal	Likert agreement	Ordering, median	Distances between levels
Interval	Temperature °C	Differences	Ratios (20° is not twice 10°)
Ratio	Income, population	Everything, including ratios	—

. . .

The mean of a Likert scale is a lie you can compute. Whether it is a useful lie is an empirical question, and one of the standing arguments in applied survey work.

Students nod along and then immediately average a Likert scale. Make it concrete:

Ask: "Is the difference between 'strongly disagree' and 'disagree' the same size as between 'neutral' and 'agree'?" They will say no. Then: "So what does the number 3.4 mean, when you average them?"

Let the silence sit. The answer is that it means something only if you assume equal spacing - which is an assumption, and it should appear in the paper.

Discrete vs. continuous

Discrete

Counts. Number of bank branches in a county. Number of loan applications.

Takes isolated values. Between 3 and 4 there is nothing.

Continuous

Measured quantities. Income, interest rate, unemployment rate.

Between any two values there is another. Limited only by your instrument.

. . .

Money is formally discrete — there are no fractional cents — but we treat it as continuous because the granularity is far finer than anything we care about. That is a modeling choice, and it is fine, as long as you know you made it.

Accuracy, precision, resolution, range

Four different things that students routinely collapse into "how good is the data."

- **Accuracy** — how close to the truth, on average. A scale that reads 3 lbs heavy is *inaccurate*.
- **Precision** — how tightly repeated measurements cluster. A scale that gives 150.1, 154.8, 147.2 is *imprecise*.
- **Resolution** — the finest distinction the instrument can record. Income binned into $10k brackets has coarse resolution.

- **Range** — what the instrument can record at all. Topcoded income is a range problem.

. . .

> You can be precisely wrong. Most bad empirical work is precisely wrong, and reports three decimal places to prove it.

Draw two dartboards on the chalkboard (press B).

Board 1: tight cluster, far from the bullseye -> precise, inaccurate. Biased. Board 2: scattered wide, centered on the bullseye -> accurate, imprecise. Noisy.

Punchline: more data fixes board 2 and does nothing for board 1. Sample size cures noise, not bias. That sentence does a lot of work later in the term.

Sample vs. population

- The **population** is every unit you want to say something about.
- The **sample** is the units you actually observe.
- Statistics is entirely about the gap between the two.

. . .

The word "population" does not mean people. It means *the set your claim is about.* Defining it is your job, and it is harder than it looks:

> "The effect of branch closures on credit access" — for whom? All US counties? Counties that had a branch to lose? Counties that lost one between 2010 and 2020? Each is a different population and a different paper.

Push on this. Make them commit to one, then ask what changes:

- All US counties -> includes counties that never had a branch. Denominator wrong.
- Counties with >=1 branch in 2010 -> conditioned on a pre-period outcome.
- Counties that lost a branch 2010-2020 -> no comparison group at all.

Do not resolve it. The point is that the population is a CHOICE with consequences, and they should feel the discomfort of making it.

The problem with $N = 1$

- A single observation has no variance, so it carries no information about variance.
- Anecdotes are $N = 1$ studies with a compelling narrator.
- "But my cousin got denied and he had good credit" is data. It is one observation of unknown provenance from a non-random draw.

. . .

This is not a reason to dismiss anecdotes. It is a reason to be precise about what they can and cannot establish. An anecdote can prove a thing is *possible*. It cannot establish how *common* it is.

The ethics of making up data

Cohen closes Chapter 2 on this, and it belongs in a first week.

- **Simulating** data and saying so: essential, and half of what we do this term.
- **Imputing** missing values and documenting it: standard practice, covered in the cleaning block.
- **Dropping** observations that hurt your result: fraud.
- **Fabricating** observations: fraud.

. . .

The line is not "did you generate numbers." It is **"can a reader reconstruct exactly what you did."** That is why every assignment in this course is a document that runs.

Reproducibility, and why it is a statistical issue

The problem with clicking

- You open a spreadsheet. You delete three rows that look wrong. You sort by income. You copy a column.
- Six weeks later a referee asks why $N = 4{,}812$ instead of 4,831.
- You have no idea.

. . .

If your analysis is a sequence of clicks, your result is not a finding. It is a rumor.

What a `.qmd` file gives you

One file containing:

1. Your **writing**
2. Your **code**
3. The **output** of that code
4. Your **figures**

. . .

When you render it, the code runs *from the top, in a clean session*. If it renders, it reproduces. If it does not render, you found a bug that you would otherwise have shipped.

. . .

> These slides are a `.qmd` file. So is every problem set you will submit. So is your final paper.

The three parts of a `.qmd`

````
---
title: "My Analysis"
author: "Your Name"
format: html
---

## Some writing

Regular text, with **bold**
and *italic* and a list:

- point one
- point two

```{r}
2 + 2
```
````

1. YAML header

Between the `---` fences. Document-level settings: title, author, output format.

2. Text

Markdown. Everything that is not a chunk.

3. Code chunks

Fenced with three backticks and `{r}` or `{python}`. This is where code lives and runs.

Markdown, in ninety seconds

You write	You get
`**bold text**`	**bold text**
`*italic text*`	*italic text*
`` `code` ``	`code`
`# Heading 1`	A large heading
`## Heading 2`	A smaller heading
`- item`	A bullet
`1. item`	A numbered list
`> quoted`	A block quote
`[text](url)`	A link

. . .

In a revealjs deck like this one, `##` starts a new slide. That is the only slide-specific rule you need today.

Chunk options

Options go on lines starting `#|` at the top of the chunk.

````
```{r}
#| label: fig-income
#| echo: false
#| fig-cap: "Median income by region"
ggplot(metros, aes(x = region, y = med_income)) + geom_col()
```
````

Option	Effect
`echo: false`	Run it, hide the code, show the output
`eval: false`	Show the code, do not run it
`include: false`	Run it, hide everything
`warning: false`	Suppress warnings
`error: true`	Let an error render instead of stopping
`label:`	Name it, so you can cross-reference it

Math mode

Quarto speaks LaTeX. This matters more than it sounds — model notation next to the code that fits it is the whole point.

Inline, one dollar sign each side:

```
The mean is $\bar{x}$ and the
variance is $s^2$.
```

The mean is $\bar{x}$ and the variance is s^2.

Display, two dollar signs:

```
$$
\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i
$$
```

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

. . .

`\bar{}` `\hat{}` `\sum` `\frac{}{}` `\sqrt{}` `\alpha` `\beta` `\sigma` `_subscript` `^superscript` — that is 90% of what you need this term.

You Try — your first document

Create `day-02-notes.qmd`. Fill in every blank, then render. **Five minutes.**

````
---
title: "____"
author: "____"
format: html
---

## About me

My name is ____, and one thing I want to be able to do
by November is ____.

## Some math

The sample mean is defined as

$$
\bar{x} = ____
$$

## Some code

```{r}
# Compute the mean of the numbers 4, 8, 15, 16, 23, 42
____
```
````

Solution — your first document

````
---
title: "Day 2 Notes"
author: "Your Name"
format: html
---

## About me

My name is [name], and one thing I want to be able to do
by November is replicate a published paper end to end.

## Some math
````

````
The sample mean is defined as

$$
\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i
$$

## Some code

```{r}
mean(c(4, 8, 15, 16, 23, 42))
```
````

Watch for: $$ on the same line as the formula (renders inline, looks wrong), and a missing blank line before the code fence. The two most common first-render failures.

Hour 2 — The tidyverse

Loading it

```
# once, ever:
install.packages("tidyverse")
```

```
library(tidyverse)
```

. . .

That one call loads **dplyr** (manipulation), **ggplot2** (plots), **tidyr** (reshaping), **readr** (import), and several more.

. . .

The startup message listing conflicts is normal. `dplyr::filter()` masks `stats::filter()`, and you want `dplyr`'s.

Our practice data

Twelve metro areas. Simulated for now — we pull the real thing at the end of the hour.

```r
metros <- tibble(
  metro = c(
    "Atlanta",
    "Baltimore",
    "Boston",
    "Charlotte",
    "Chicago",
    "Dallas",
    "Detroit",
    "Houston",
    "Memphis",
    "Phoenix",
    "Seattle",
    "Washington"
  ),
  region = c(
    "South",
    "South",
    "Northeast",
    "South",
    "Midwest",
    "South",
    "Midwest",
    "South",
    "South",
    "West",
    "West",
    "South"
  ),
  population = c(6.1, 2.8, 4.9, 2.7, 9.5, 7.6, 4.3, 7.1, 1.3, 4.9, 4.0, 6.3) *
    1e6,
  med_income = c(
    76500,
    91200,
    106800,
    78900,
    84300,
    82100,
    68400,
    75800,
    59200,
    76100,
```

```
    105300,
    118600
  ),
  unemployment = c(3.4, 3.9, 3.1, 3.3, 4.2, 3.6, 4.8, 4.1, 4.6, 3.5, 3.8, 2.9)
)
```

Look before you leap

Two functions you should run on **every** dataset, before anything else.

```
glimpse(metros)
#> Rows: 12
#> Columns: 5
#> $ metro        <chr> "Atlanta", "Baltimore", "Boston", "Charlotte", "Chicago",~
#> $ region       <chr> "South", "South", "Northeast", "South", "Midwest", "South~
#> $ population   <dbl> 6100000, 2800000, 4900000, 2700000, 9500000, 7600000, 430~
#> $ med_income   <dbl> 76500, 91200, 106800, 78900, 84300, 82100, 68400, 75800, ~
#> $ unemployment <dbl> 3.4, 3.9, 3.1, 3.3, 4.2, 3.6, 4.8, 4.1, 4.6, 3.5, 3.8, 2.9
```

And a numerical summary

```
summary(metros)
#>        metro              region     population        med_income
#>  Length   :12   Length   :12   Min.   :1300000   Min.   : 59200
#>  N.unique :12   N.unique : 4   1st Qu.:3700000   1st Qu.: 76025
#>  N.blank  : 0   N.blank  : 0   Median :4900000   Median : 80500
#>  Min.nchar: 6   Min.nchar: 4   Mean   :5125000   Mean   : 85267
#>  Max.nchar:10   Max.nchar: 9   3rd Qu.:6500000   3rd Qu.: 94725
#>                                Max.   :9500000   Max.   :118600
#>   unemployment
#>  Min.   :2.900
#>  1st Qu.:3.375
#>  Median :3.700
#>  Mean   :3.767
#>  3rd Qu.:4.125
#>  Max.   :4.800
```

. . .

`glimpse()` answers *what are the columns and their types*. `summary()` answers *what is the range and are there NAs*. Together they catch most import disasters in ten seconds.

The pipe

`|>` takes what is on the left and passes it as the **first argument** of what is on the right.

Without the pipe — read inside out:

```
head(arrange(filter(metros, region == "South"), med_income))
```

With the pipe — read left to right:

```
metros |>
  filter(region == "South") |>
  arrange(med_income) |>
  head()
```

. . .

Read `|>` out loud as **"and then."** Take metros, *and then* keep the South, *and then* sort by income, *and then* show the top.

You will also see `%>%` from the `magrittr` package. Nearly identical; `|>` is built into R and is what we use.

`select()` — choose columns

```
metros |>
  select(metro, med_income)
#> # A tibble: 12 x 2
#>    metro      med_income
#>    <chr>           <dbl>
#>  1 Atlanta         76500
#>  2 Baltimore       91200
#>  3 Boston         106800
#>  4 Charlotte       78900
#>  5 Chicago         84300
#>  6 Dallas          82100
#>  7 Detroit         68400
```

```
#>  8 Houston         75800
#>  9 Memphis         59200
#> 10 Phoenix         76100
#> 11 Seattle        105300
#> 12 Washington     118600
```

`select()` — more ways

```
metros |> select(metro, region, unemployment) # several
metros |> select(-population) # all but one
metros |> select(metro:population) # a range
metros |> select(starts_with("med")) # by name pattern
metros |> select(where(is.numeric)) # by type
```

You Try — `select()`

Select **only** the metro name and the unemployment rate.

```
metros |>
  ____
```

. . .

Then: select every column **except** population, using a minus sign.

```
metros |>
  ____
```

Three minutes.

Solution — `select()`

```
metros |> select(metro, unemployment)
#> # A tibble: 12 x 2
#>    metro      unemployment
#>    <chr>             <dbl>
#>  1 Atlanta             3.4
```

```
#>  2 Baltimore           3.9
#>  3 Boston              3.1
#>  4 Charlotte           3.3
#>  5 Chicago             4.2
#>  6 Dallas              3.6
#>  7 Detroit             4.8
#>  8 Houston             4.1
#>  9 Memphis             4.6
#> 10 Phoenix             3.5
#> 11 Seattle             3.8
#> 12 Washington          2.9
```

Solution — `select()`, part two

```
metros |> select(-population)
#> # A tibble: 12 x 4
#>    metro      region    med_income unemployment
#>    <chr>      <chr>          <dbl>        <dbl>
#>  1 Atlanta    South          76500          3.4
#>  2 Baltimore  South          91200          3.9
#>  3 Boston     Northeast     106800          3.1
#>  4 Charlotte  South          78900          3.3
#>  5 Chicago    Midwest        84300          4.2
#>  6 Dallas     South          82100          3.6
#>  7 Detroit    Midwest        68400          4.8
#>  8 Houston    South          75800          4.1
#>  9 Memphis    South          59200          4.6
#> 10 Phoenix    West           76100          3.5
#> 11 Seattle    West          105300          3.8
#> 12 Washington South         118600          2.9
```

Worth mentioning: select() also REORDERS. select(metros, region, metro) returns the columns in that order. Students discover this by accident later; naming it now saves confusion.

`filter()` — choose rows

```
metros |>
  filter(med_income > 90000)
```

```
#> # A tibble: 4 x 5
#>   metro      region    population med_income unemployment
#>   <chr>      <chr>          <dbl>      <dbl>        <dbl>
#> 1 Baltimore  South        2800000      91200          3.9
#> 2 Boston     Northeast    4900000     106800          3.1
#> 3 Seattle    West         4000000     105300          3.8
#> 4 Washington South        6300000     118600          2.9
```

. . .

`=` assigns. `==` tests equality. `filter(region = "South")` is an error; `filter(region == "South")` is what you meant. You will make this mistake. Everyone does.

Comparison operators

Operator	Means
`==`	equal to
`!=`	not equal to
`> >= < <=`	greater / less than
`%in%`	is one of
`&`	and
`|`	or
`!`	not
`is.na()`	is missing

```
metros |> filter(region %in% c("South", "West"))
metros |> filter(unemployment < 3.5 & med_income > 80000)
```

You Try — `filter()`

Keep only metros in the **Midwest**.

```
metros |>
  ____
```

. . .

Keep metros with unemployment **below 3.5** *and* median income **above $80,000**.

```
metros |>
  ____
```

Four minutes.

Solution — `filter()`

```
metros |> filter(region == "Midwest")
#> # A tibble: 2 x 5
#>   metro   region  population med_income unemployment
#>   <chr>   <chr>        <dbl>      <dbl>        <dbl>
#> 1 Chicago Midwest    9500000      84300          4.2
#> 2 Detroit Midwest    4300000      68400          4.8
```

Solution — `filter()`, compound

```
metros |> filter(unemployment < 3.5 & med_income > 80000)
#> # A tibble: 2 x 5
#>   metro      region    population med_income unemployment
#>   <chr>      <chr>          <dbl>      <dbl>        <dbl>
#> 1 Boston     Northeast    4900000     106800          3.1
#> 2 Washington South        6300000     118600          2.9
```

Only Boston, Seattle and Washington survive. Ask what happens with | instead of & - the answer jumps to nine rows, and the difference between "and" and "or" stops being abstract.

The = vs == mistake

Let them hit it first. Then show it:

```
metros |> filter(region = "South")
#> Error in `filter()`:
#> ! We detected a named input.
#> i This usually means that you've used `=` instead of `==`.
#> i Did you mean `region == "South"`?
```

. . .

The error message is unusually good — R actually guesses what you meant. Most R errors are not this kind. Build the habit now: **read the whole error message before changing anything.**

`mutate()` — make new columns

```
metros |>
  mutate(pop_millions = population / 1e6) |>
  select(metro, population, pop_millions)
#> # A tibble: 12 x 3
#>    metro      population pop_millions
#>    <chr>           <dbl>        <dbl>
#>  1 Atlanta       6100000          6.1
#>  2 Baltimore     2800000          2.8
#>  3 Boston        4900000          4.9
#>  4 Charlotte     2700000          2.7
#>  5 Chicago       9500000          9.5
#>  6 Dallas        7600000          7.6
#>  7 Detroit       4300000          4.3
#>  8 Houston       7100000          7.1
#>  9 Memphis       1300000          1.3
#> 10 Phoenix       4900000          4.9
#> 11 Seattle       4000000          4
#> 12 Washington    6300000          6.3
```

`mutate()` — a real one

Income relative to the sample average, as a percentage:

```
metros |>
  mutate(income_index = 100 * med_income / mean(med_income)) |>
  select(metro, med_income, income_index) |>
  arrange(desc(income_index))
#> # A tibble: 12 x 3
#>    metro      med_income income_index
#>    <chr>           <dbl>        <dbl>
#>  1 Washington     118600        139.
#>  2 Boston         106800        125.
#>  3 Seattle        105300        123.
```

```
#>  4 Baltimore       91200        107.
#>  5 Chicago         84300         98.9
#>  6 Dallas          82100         96.3
#>  7 Charlotte       78900         92.5
#>  8 Atlanta         76500         89.7
#>  9 Phoenix         76100         89.2
#> 10 Houston         75800         88.9
#> 11 Detroit         68400         80.2
#> 12 Memphis         59200         69.4
```

You Try — `mutate()`

Create a column `emp_rate` equal to `100 - unemployment`.

```
metros |>
  mutate(____) |>
  select(metro, unemployment, emp_rate)
```

. . .

Then create `income_rank` using `min_rank(desc(med_income))`, ranking the metros from richest to poorest.

```
metros |>
  mutate(____) |>
  arrange(income_rank)
```

Five minutes.

Solution — `mutate()`

```
metros |>
  mutate(emp_rate = 100 - unemployment) |>
  select(metro, unemployment, emp_rate)
#> # A tibble: 12 x 3
#>    metro      unemployment emp_rate
#>    <chr>             <dbl>    <dbl>
#>  1 Atlanta             3.4     96.6
#>  2 Baltimore           3.9     96.1
```

```
#>  3 Boston              3.1     96.9
#>  4 Charlotte           3.3     96.7
#>  5 Chicago             4.2     95.8
#>  6 Dallas              3.6     96.4
#>  7 Detroit             4.8     95.2
#>  8 Houston             4.1     95.9
#>  9 Memphis             4.6     95.4
#> 10 Phoenix             3.5     96.5
#> 11 Seattle             3.8     96.2
#> 12 Washington          2.9     97.1
```

Solution — `mutate()` with a rank

```
metros |>
  mutate(income_rank = min_rank(desc(med_income))) |>
  select(metro, med_income, income_rank) |>
  arrange(income_rank)
#> # A tibble: 12 x 3
#>    metro      med_income income_rank
#>    <chr>           <dbl>       <int>
#>  1 Washington     118600           1
#>  2 Boston         106800           2
#>  3 Seattle        105300           3
#>  4 Baltimore       91200           4
#>  5 Chicago         84300           5
#>  6 Dallas          82100           6
#>  7 Charlotte       78900           7
#>  8 Atlanta         76500           8
#>  9 Phoenix         76100           9
#> 10 Houston         75800          10
#> 11 Detroit         68400          11
#> 12 Memphis         59200          12
```

`summarise()` — collapse to one row

```
metros |>
  summarise(
    n = n(),
```

```
    mean_income = mean(med_income),
    sd_income = sd(med_income)
  )
#> # A tibble: 1 x 3
#>       n mean_income sd_income
#>   <int>       <dbl>     <dbl>
#> 1    12      85267.    17261.
```

. . .

On its own, `summarise()` is not very interesting. Paired with `group_by()`, it is most of applied data work.

`group_by()` + `summarise()`

```
metros |>
  group_by(region) |>
  summarise(
    n = n(),
    mean_income = mean(med_income),
    mean_unemp = mean(unemployment)
  ) |>
  arrange(desc(mean_income))
#> # A tibble: 4 x 4
#>   region        n mean_income mean_unemp
#>   <chr>     <int>       <dbl>      <dbl>
#> 1 Northeast     1     106800        3.1
#> 2 West          2      90700        3.65
#> 3 South         7      83186.       3.69
#> 4 Midwest       2      76350        4.5
```

You Try — group and summarise

For each **region**, compute the number of metros, the *median* income, and the *maximum* unemployment rate.

```
metros |>
  group_by(____) |>
  summarise(
    n = ____,
```

```
    median_income = ____,
    max_unemp = ____
  )
```

. . .

Then answer in a sentence, in your `.qmd`: which region looks best off, and what does this table hide?

Six minutes.

Solution — group and summarise

```
metros |>
  group_by(region) |>
  summarise(
    n = n(),
    median_income = median(med_income),
    max_unemp = max(unemployment)
  ) |>
  arrange(desc(median_income))
#> # A tibble: 4 x 4
#>   region        n median_income max_unemp
#>   <chr>     <int>         <dbl>     <dbl>
#> 1 Northeast     1        106800       3.1
#> 2 West          2         90700       3.8
#> 3 South         7         78900       4.6
#> 4 Midwest       2         76350       4.8
```

What that table hides

1. **Northeast has $n = 1$.** Its "median" is Boston. This is the $N = 1$ slide from Hour 1, arriving one hour later in your own output.
2. **It is unweighted by population.** Memphis (1.3M) counts as much as Houston (7.1M). A regional median that ignores population answers a different question than most people assume.
3. **Within-region variance is invisible.** The South spans Memphis at \$59,200 and Washington at \$118,600 — a range wider than the gap between any two regional medians.

. . .

Point 3 is why Thursday is about **distributions**, not summaries.

Getting only #1 is a good outcome for week one. Supply #2 and #3 yourself.

Inline code

You can put R directly in a sentence, so numbers in your prose never go stale.

```
Our sample contains 12 metro areas with a mean
median income of \ensuremath{8.5267\times 10^{4}} dollars.
```

Renders as:

> Our sample contains 12 metro areas with a mean median income of 8.5267×10^4 dollars.

. . .

Never type a number into your text that came out of your analysis. Compute it inline. When the data change, the sentence changes with it — and that is a *correctness* property, not a convenience.

ggplot2

The grammar

Every plot is three things:

1. **Data** — the data frame
2. **Aesthetics** — which variable maps to which visual property (`x`, `y`, `color`, `size`)
3. **Geometry** — what marks to draw (`geom_point`, `geom_col`, `geom_histogram`)

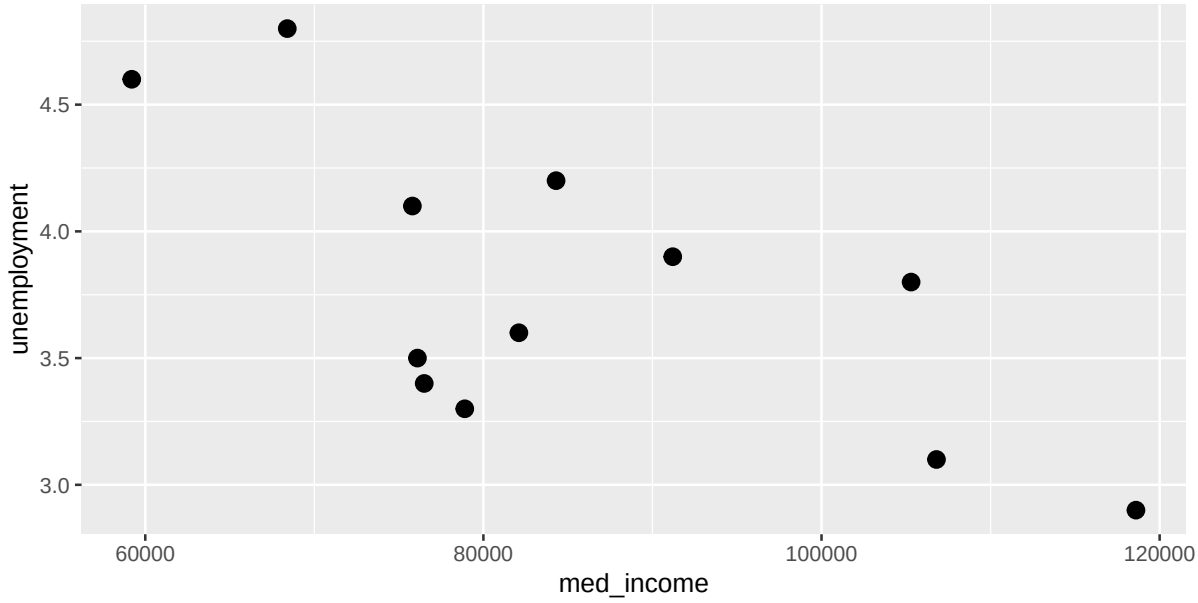
```
ggplot(data, aes(x = ..., y = ...)) +
  geom_something()
```

. . .

Layers are joined with `+`, not `|>`. This is a historical wart. Everyone trips on it.

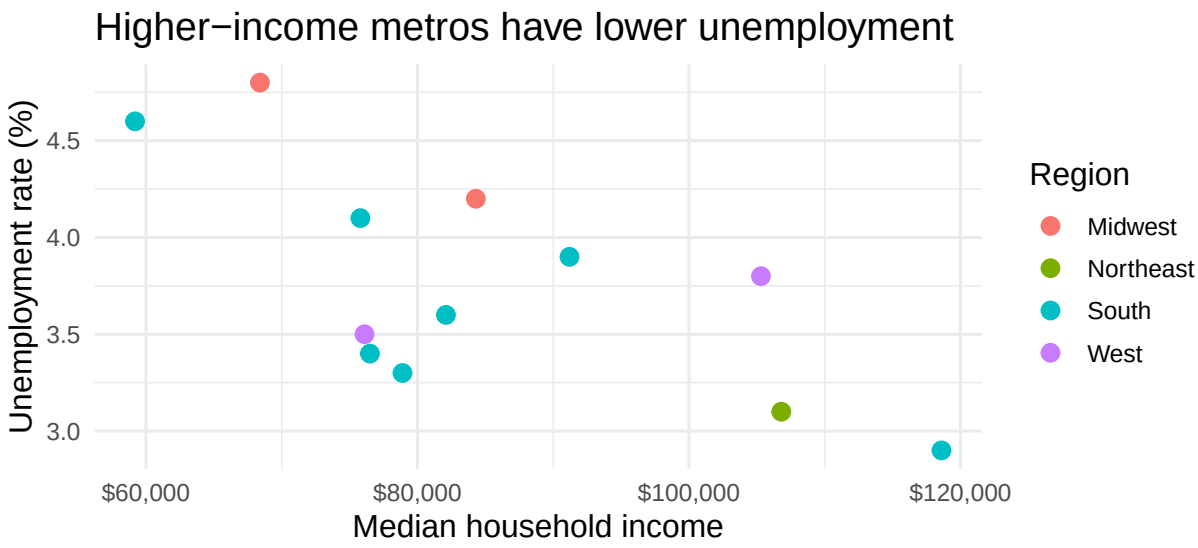
A scatterplot

```
ggplot(metros, aes(x = med_income, y = unemployment)) +
  geom_point(size = 3)
```



Adding color and labels

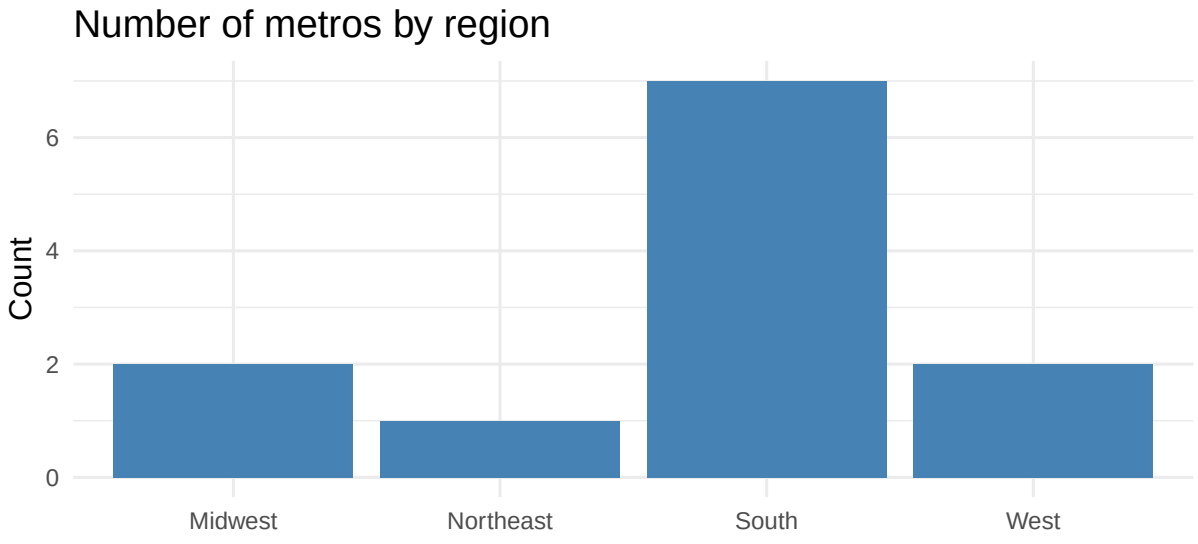
```
ggplot(metros, aes(x = med_income, y = unemployment, color = region)) +
  geom_point(size = 3) +
  scale_x_continuous(labels = scales::dollar) +
  labs(
    title = "Higher-income metros have lower unemployment",
    x = "Median household income",
    y = "Unemployment rate (%)",
    color = "Region"
  ) +
  theme_minimal(base_size = 13)
```



Bars from raw rows: `geom_bar()`

`geom_bar()` **counts** for you.

```
ggplot(metros, aes(x = region)) +
  geom_bar(fill = "steelblue") +
  labs(title = "Number of metros by region", x = NULL, y = "Count") +
  theme_minimal(base_size = 13)
```

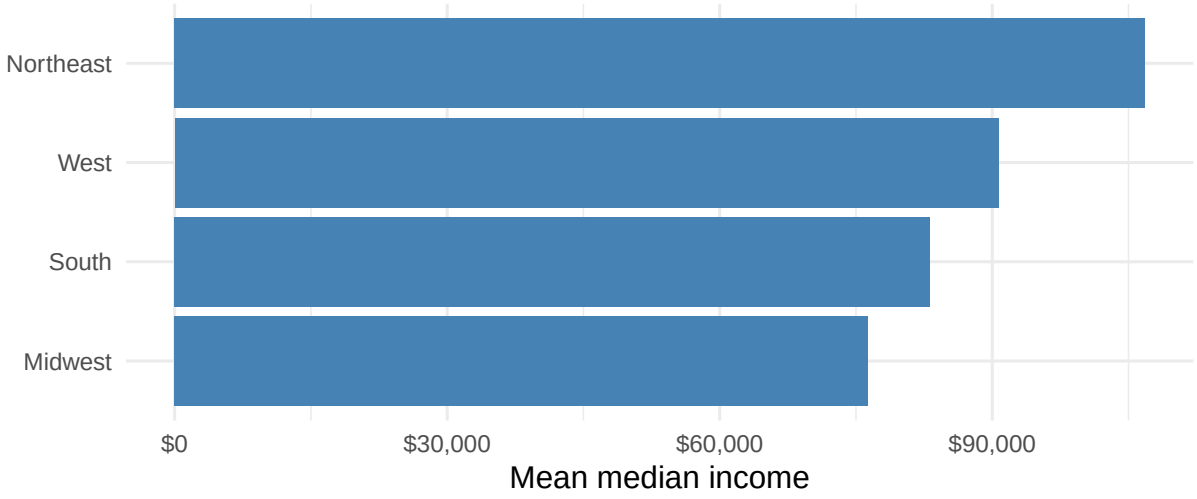


Bars from a summary: `geom_col()`

`geom_col()` plots the value you give it. Summarise first, then plot.

```
region_income <- metros |>
  group_by(region) |>
  summarise(mean_income = mean(med_income))

ggplot(
  region_income,
  aes(x = fct_reorder(region, mean_income), y = mean_income)
) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  scale_y_continuous(labels = scales::dollar) +
  labs(x = NULL, y = "Mean median income") +
  theme_minimal(base_size = 13)
```



`geom_bar()` vs `geom_col()`

> **`geom_bar()`** takes raw rows and counts them. **`geom_col()`** takes a number you already computed and draws it at that height.
> Nine out of ten "why is my bar chart wrong" problems are using one when you wanted the other.

You Try — a plot

Make a scatterplot of **population** (x) against **median income** (y), colored by region, with a title and axis labels.

```
ggplot(metros, aes(x = ____, y = ____, color = ____)) +
  geom_point(size = 3) +
  labs(
    title = "____",
    x = "____",
    y = "____"
  ) +
  theme_minimal()
```

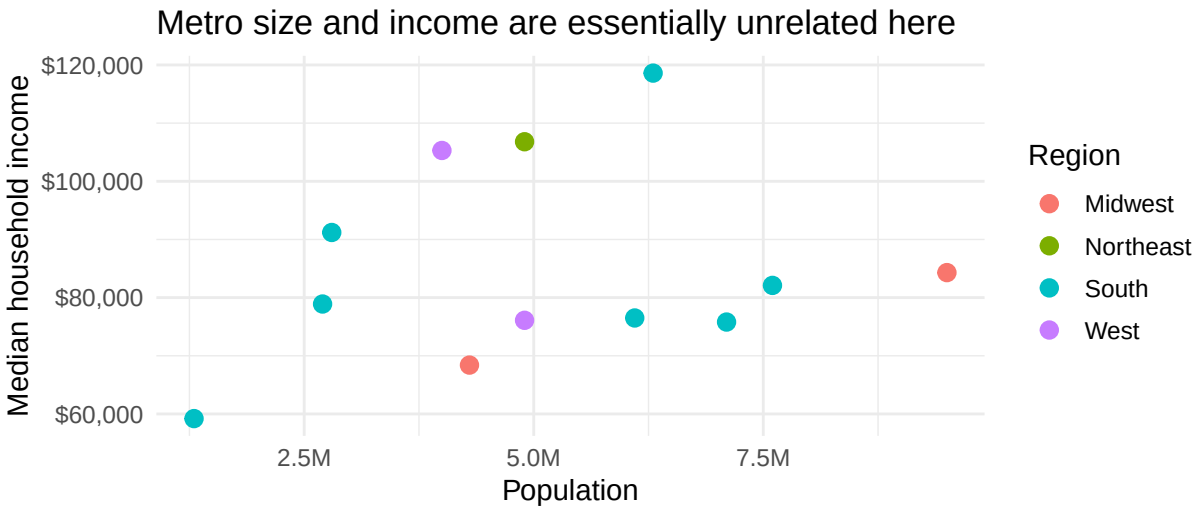
. . .

Then: does the relationship look like anything? Write one sentence about what you see — including "nothing" if that is the honest answer.

Six minutes.

Solution — a plot

```
ggplot(metros, aes(x = population, y = med_income, color = region)) +
  geom_point(size = 3) +
  scale_x_continuous(
    labels = scales::label_number(scale = 1e-6, suffix = "M")
  ) +
  scale_y_continuous(labels = scales::dollar) +
  labs(
    title = "Metro size and income are essentially unrelated here",
    x = "Population",
    y = "Median household income",
    color = "Region"
  ) +
  theme_minimal(base_size = 12)
```



What to say about that plot

> The honest answer is **"nothing."** There is no visible relationship, and with $n = 12$ there is no power to detect one anyway.

- "I don't see a pattern" is the correct scientific answer, and most people are reluctant to give it.
- But could there be a relationship you cannot see with twelve points? **Yes.** Absence of evidence is not evidence of absence.
- Drawing a trend line through this would be fitting noise. We will name that on 10/8.

If the student draws a geom_smooth() through it anyway, show it, discuss overfitting, then delete it.

Real data

The Census API

Everything so far has been numbers I made up. Here is the real thing.

```
library(tidycensus)

# one time only, with the key you requested last week:
census_api_key("YOUR_KEY_HERE", install = TRUE)
```

. . .

```
# Median household income, by state, ACS 5-year 2022
income_by_state <- get_acs(
  geography = "state",
  variables = "B19013_001", # median household income
  year = 2022,
  survey = "acs5"
)

glimpse(income_by_state)
```

What comes back

```
Rows: 52
Columns: 5
$ GEOID    <chr> "01", "02", "04", "05", "06", ...
$ NAME     <chr> "Alabama", "Alaska", "Arizona", ...
$ variable <chr> "B19013_001", "B19013_001", ...
$ estimate <dbl> 59609, 86370, 72581, 55432, 91905, ...
$ moe      <dbl> 609, 2454, 654, 700, 447, ...
```

. . .

Note that `moe` column

The ACS is a **sample**, so every estimate carries a margin of error. Most people who use this data ignore that column. You will not — it is the sample-vs-population gap from Hour 1, printed right next to the number.

You Try — your first real pull

Pull median household income for **counties in Maryland**, then find the five highest.

```
md_income <- get_acs(
  geography = "____",
  variables = "B19013_001",
  state = "____",
  year = 2022,
  survey = "acs5"
```

```
)

md_income |>
  arrange(____) |>
  head(5) |>
  select(NAME, estimate, moe)
```

. . .

Then: find two counties whose estimates differ by less than the sum of their margins of error. What can you say about their ranking?

Eight minutes.

Solution — your first real pull

```
md_income <- get_acs(
  geography = "county",
  variables = "B19013_001",
  state = "MD",
  year = 2022,
  survey = "acs5"
)

md_income |>
  arrange(desc(estimate)) |>
  head(5) |>
  select(NAME, estimate, moe)
```

```
# A tibble: 5 × 3
  NAME                           estimate   moe
1 Howard County, Maryland          129549  3288
2 Calvert County, Maryland         124847  6320
3 Montgomery County, Maryland      122903  1477
4 Charles County, Maryland         110716  4623
5 Anne Arundel County, Maryland    110255  2314
```

Exact figures shift with the ACS vintage — read your own output rather than matching these.

The margin-of-error question

Howard: \$129,549 ± \$3,288. Calvert: \$124,847 ± \$6,320.

- Difference in point estimates: **\$4,702**.
- Combined uncertainty: comfortably **larger** than that.
- So the ranking between them is **not statistically distinguishable**. "Howard is the richest county in Maryland" is a claim these data do not support against Calvert.

. . .

> Every "top 10 richest counties" list you have ever seen ignores this. That is the difference between a fact and an artifact of sampling noise. In ten weeks you will be computing these intervals yourself.

Mini-exercise — put it together

In `day-02-notes.qmd`, using `metros`:

1. Create a column `income_per_100k_pop` = `med_income / (population / 1e5)`
2. Group by region and report the mean of that column
3. Make a `geom_col()` chart of the result, sorted, with a dollar-formatted axis
4. Write two sentences: what does this measure, and why is it a slightly silly quantity?

. . .

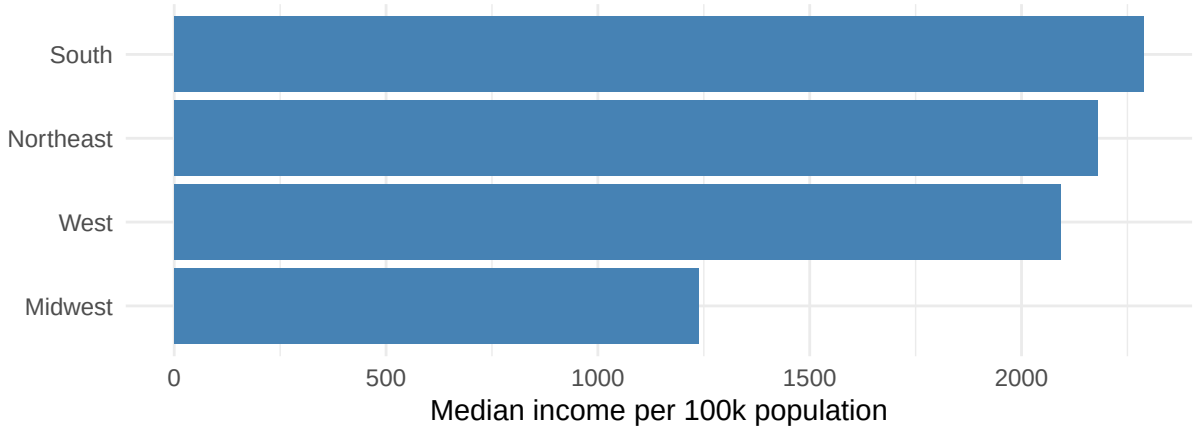
> Question 4 is the real question. Being able to compute something is not a reason to compute it.

Solution — mini-exercise

```
region_summary <- metros |>
  mutate(income_per_100k_pop = med_income / (population / 1e5)) |>
  group_by(region) |>
  summarise(mean_ratio = mean(income_per_100k_pop))

ggplot(
  region_summary,
  aes(x = fct_reorder(region, mean_ratio), y = mean_ratio)
) +
```

```
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(x = NULL, y = "Median income per 100k population") +
  theme_minimal(base_size = 12)
```



Why the quantity is silly

> The chart is competent and the number is meaningless.

- **The units are nonsense.** Dollars per person per hundred thousand people. There is no question this answers.
- **It is dominated by the denominator.** Memphis has the lowest income *and* the smallest population, so it ranks near the top. The measure rewards being small far more than being rich.
- **It confuses a rate with a level.** Median income is already a per-person quantity. Dividing by population again does not "adjust" for anything.

. . .

`mutate()` will compute anything you ask it to. Nothing in the toolchain will tell you the result is meaningless. That judgment is yours, and it is the part that does not automate.

Checklist before you go

You should now be able to:

- ☐ Explain what a `.qmd` file does and name its three parts
- ☐ Set chunk options with `#|`

- ☐ Write inline and display math
- ☐ Use `select()`, `filter()`, `mutate()`, `group_by()`, `summarise()`
- ☐ Chain them with `|>`
- ☐ Build a plot from data + aesthetics + geometry
- ☐ Tell `geom_bar()` from `geom_col()`
- ☐ Pull a real series from the Census API

. . .

Thursday: visualization proper — Cohen Ch. 3. Histograms, box plots, and how to lie with an axis. **Problem Set 1 is assigned Thursday, due Monday 9/21.**

See you Thursday

Read Cohen Chapter 3 · Bring `day-02-notes.qmd`